KC#5

Start Assignment

Due Wednesday by 11:59pm **Points** 10 **Submitting** a text entry box or a file upload **Available** until Jul 15 at 11:59pm

Homework Practice (Complete Before Next Class)

HW1 (Medium) — Registration Form Data-Driven Test

Pick a target site that has a registration form (suggestions: **https://practice.expandtesting.com/register (https://practice.expandtesting.com/register)** , **https://demoqa.com/automation-practice-form (https://demoqa.com/automation-practice-form)** , or **https://automationexercise.com (https://automationexercise.com/)**).

Build a `RegistrationPage` (extending `BasePage`, locators as `static` fields, methods describe intent — same conventions as Week 1). Then create `data/registration-data.xlsx` with **20 rows**.

Required columns: `testId`, `testCase`, `firstName`, `lastName`, `email`, `password`, `confirmPassword`, `expected`, `expectedError` (or similar).

Required mix:

- At least 5 valid registrations (vary first/last name shape, password content)
- At least 5 invalid emails (missing @, multiple @, no domain, no TLD, etc.)
- At least 3 password-mismatch cases (passwords don't match confirm)
- At least 2 password length boundaries (min - 1, min, max, max + 1 — pick which match the form)
- At least 2 missing-required-field cases
- At least 2 special-char-in-name cases (apostrophes, hyphens, accented chars, emoji)

Write `tests/registration.dd.spec.js` using the same pattern as the login test. Each row's `expected` column drives whether you assert success or a specific error.

Push to GitHub. Submit the repo link.

HW2 (Medium) — Source Switch

Make HW1 work from EITHER an Excel or a CSV file by reading an env var:

```
const source = process.env.DATA_SOURCE || "xlsx";
const dataset = source === "csv" ? readCsv("data/registration-data.csv") : readExcel("data/registration-data.xlsx");
```

Provide both `data/registration-data.xlsx` and `data/registration-data.csv` (same content). Verify `DATA_SOURCE=csv npm test` and `DATA_SOURCE=xlsx npm test` both produce 20 results.

HW3 (Medium) — Filtering

Add support for running only a subset of rows by `testId` prefix. E.g.:

```
TEST_FILTER=REG_VAL npm test   # only rows whose testId starts with REG_VAL
```

In your spec file:

```
const filter = process.env.TEST_FILTER || "";
const filtered = dataset.filter(row => row.testId.startsWith(filter));
filtered.forEach(row => it(...));
```

Useful for fast iteration: "I only care about validation rows right now."

HW4 (Harder) — A Helper That Handles Both

Refactor: replace `readExcel` and `readCsv` with one `readData(filepath)` in `utils/data.js` that picks the right parser based on the file extension.

```
function readData(filepath) {
    if (filepath.endsWith(".xlsx") || filepath.endsWith(".xls")) {
        return readExcel(filepath);
    }
    if (filepath.endsWith(".csv")) {
        return readCsv(filepath);
    }
    throw new Error(`Unsupported data file: ${filepath}`);
}
```

Update both your login and registration specs to use `readData`. The tests shouldn't care which file format you used.

HW5 (Stretch) — Data Validation

Before running the tests, validate the dataset itself:

- Every row has a non-empty `testId`
- Every `testId` is unique (no duplicates)
- Every row has a non-empty `expected` value of either `pass` or `fail`
- Every `fail` row has a non-empty `expectedError`

If any row fails validation, throw a clear error and skip running tests. This catches bad spreadsheet edits before you waste 3 minutes running 20 broken tests.

```
function validate(rows) {
    const ids = new Set();
    rows.forEach((row, i) => {
        if (!row.testId) throw new Error(`Row ${i + 2}: missing testId`);
        if (ids.has(row.testId)) throw new Error(`Duplicate testId: ${row.testId}`);
        ids.add(row.testId);
        if (!["pass", "fail"].includes(row.expected)) {
            throw new Error(`Row ${row.testId}: expected must be pass|fail, got ${row.expected}`);
        }
        if (row.expected === "fail" && !row.expectedError) {
            throw new Error(`Row ${row.testId}: fail rows need expectedError`);
        }
    });
}
```

(Row index `i + 2` because spreadsheets are 1-indexed and row 1 is the header.)